DATA 260 – Homework 6 Report

Student ID: 018322837 **Name: Shashank Ranjan**

Section 0 – Personal Configuration and Domain

Name	Definition	Value
SID	Student ID	018322837
SID4	Last 4 of Student ID	2837
PORT_BASE	8000 + SID mod 900(137)	8137
prefix	"s" + SID4	s2837
Seed	SID4	2837
verify sid	260000 + SID4	262837
DOMAIN_ID	SID4 MOD 8	5
Assigned Domain	Assignment domain	**Local restaurant inspections**

Repository: https://github.com/ShankySJSU/Data260-2837
Submission Tag: hw6

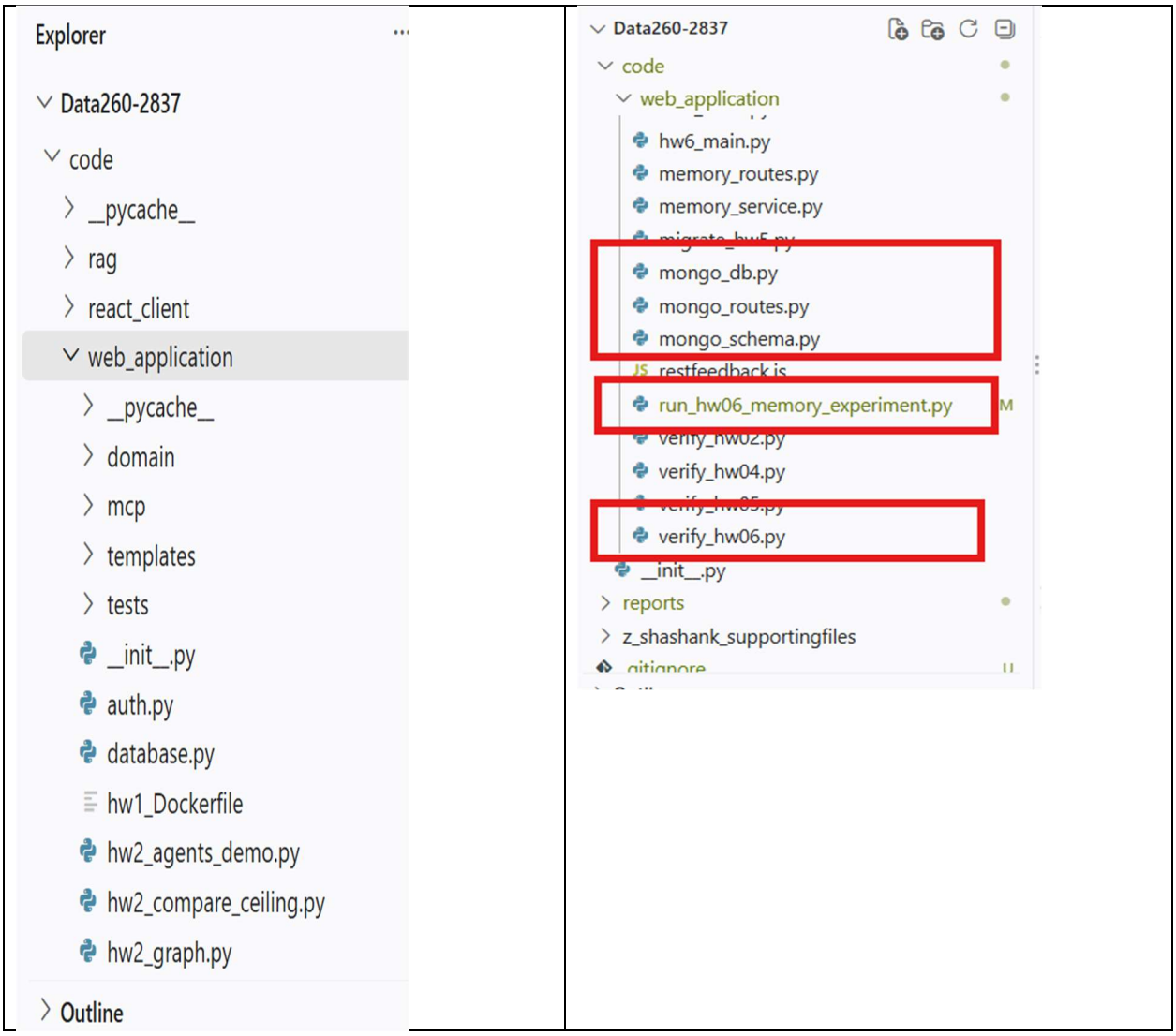
Tagged commit hash (hw6): ab5c438f34b1da0cd3495cd4ade66ff9651c3ff2
(Shashank: always use "git rev-parse HEAD"
Collaborators with access: Sbnikitha, supriyaselvanganesan

Hardware:
Local Windows PC, Python 3.12, VS Code Desktop, i7 CPU, 16GB RAM.

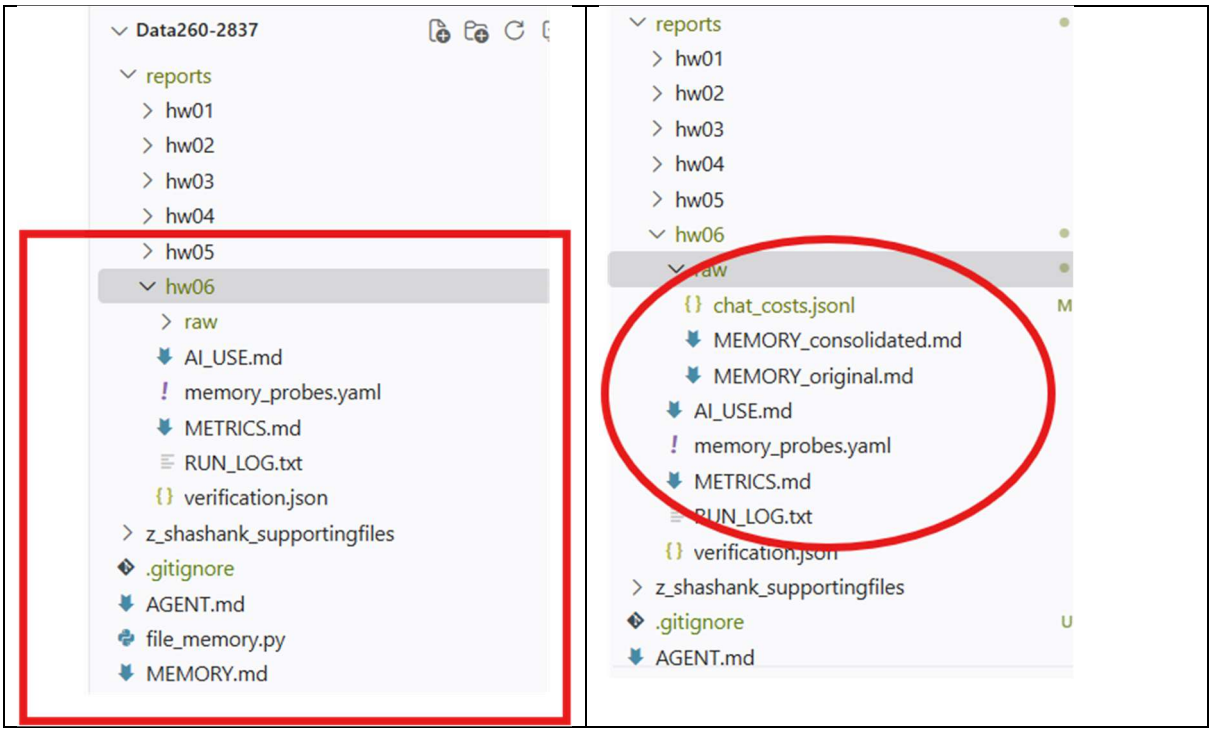
Model Used:

- Embeddings: sentence-transformers/all-MiniLM-L6-v2
- Ollama (qwen2.5:1.5b, qwen3:4b, llama3.2:1b)

Base Directory Structure:



New and changed HW6 files are boxed in red. The code is an extension of the existing HW1–HW5 codebase (no separate copy). HW6 deliverables are under reports/hw06/ (RUN_LOG.txt, raw/, METRICS.md, AI_USE.md, memory_probes.yaml, verification.json, original and consolidated MEMORY.md); AGENT.md and MEMORY.md hold the file-based memory.



Part 1 – MongoDB Document API

1.1 Database and schema

The document view of the restaurant-inspection domain lives in MongoDB database **s2837_doc** (PREFIX + "_doc"), collection **inspections**. The API runs on PORT_BASE 8137 (uvicorn hw6_main:app --port 8137). Pydantic schemas enforce validation before anything reaches MongoDB.

Field	Type / validation rule	Example from run
inspection_code	string, required, max 100 characters	HW6-INSP-001
description	string, optional	Routine restaurant inspection
status	enum: PASS, WARNING, FAIL (values used in testing) All Passed	PASS
category	enum, fixed set: Routine, Follow-up All passed	Routine
inspection_date	datetime, required (ISO format)	2026-10-18T00:00:00
created_at / updated_at	set automatically by the server (UTC); updated_at refreshed on PUT	2026-10-19T17:48:41.769
id	MongoDB ObjectId returned as string	6ad657f96238e5098b3fe8f8

1.2 API implementation

FastAPI exposes full CRUD plus a schema-status endpoint. Code is split into server configuration (CORS and JSON body parsing), the PyMongo connection, Pydantic models, and route handlers. [

Method	Route	Purpose
POST	/api/documents	Create an inspection document
GET	/api/documents	Retrieve all documents
GET	/api/documents/{id}	Retrieve one document by ID
PUT	/api/documents/{id}	Update a document
DELETE	/api/documents/{id}	Delete a document
GET	/api/documents/schema/status	Report database and collection in use

1.3 Evidence – CRUD operations

Figure 1 – MongoDB Compass connected to the local server, showing database s2837_doc and collection inspections.

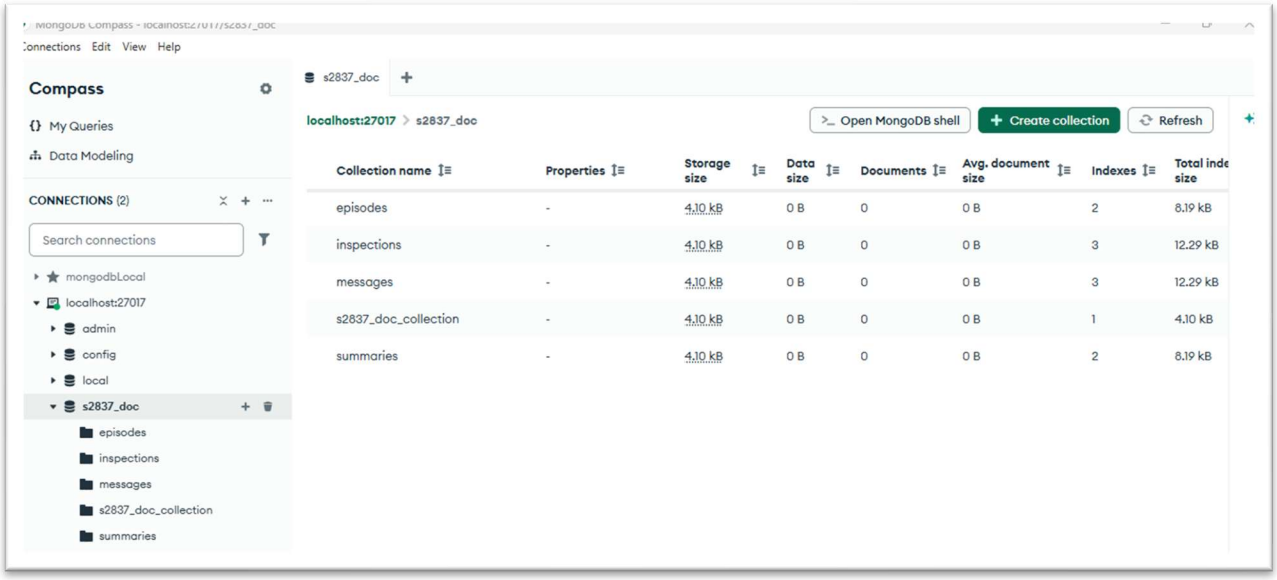


Figure 2 – Starting the API with uvicorn on port 8137 (name echoed in the terminal).

```
C:\SJSURepo\Data260\Data260-2837\code\web_application>echo "Shashank Ranjan Homework 6"
"Shashank Ranjan Homework 6"

C:\SJSURepo\Data260\Data260-2837\code\web_application>uvicorn hw6_main:app --reload --port 8137
INFO:     Will watch for changes in these directories: ['C:\\SJSURepo\\Data260\\Data260-2837\\code\\web_application']
INFO:     Uvicorn running on http://127.0.0.1:8137 (Press CTRL+C to quit)
INFO:     Started reloader process [8800] using StatReload
C:\Python\python3.12\Lib\site-packages\pydantic\_internal\_config.py:386: UserWarning: Valid config keys have changed in V2:
* 'orm_mode' has been renamed to 'from_attributes'
  warnings.warn(message, UserWarning)
INFO:     Started server process [3304]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Figure 3 – GET /api/documents/schema/status returns database s2837_doc and collection inspections.

http://localhost:8137/api/documents/schema/status

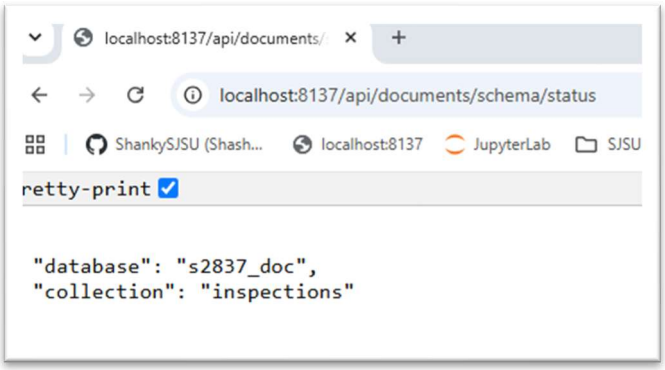


Figure 4 – POST /api/documents creates inspection HW6-INSP-001 (Postman). The response adds id, created_at and updated_at.

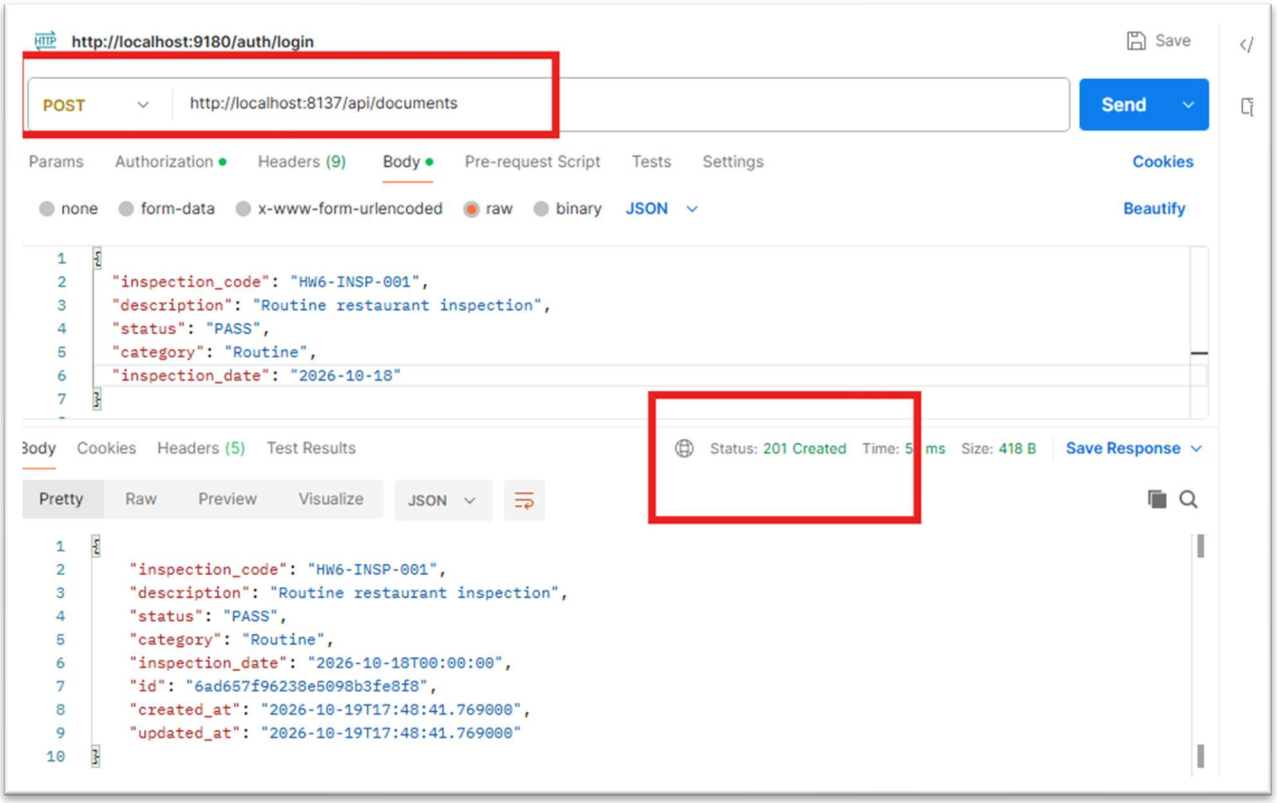


Figure 5 – GET /api/documents/{id} returns the single document 6ad657f96238e5098b3fe8f8 (HW6-INSP-001).

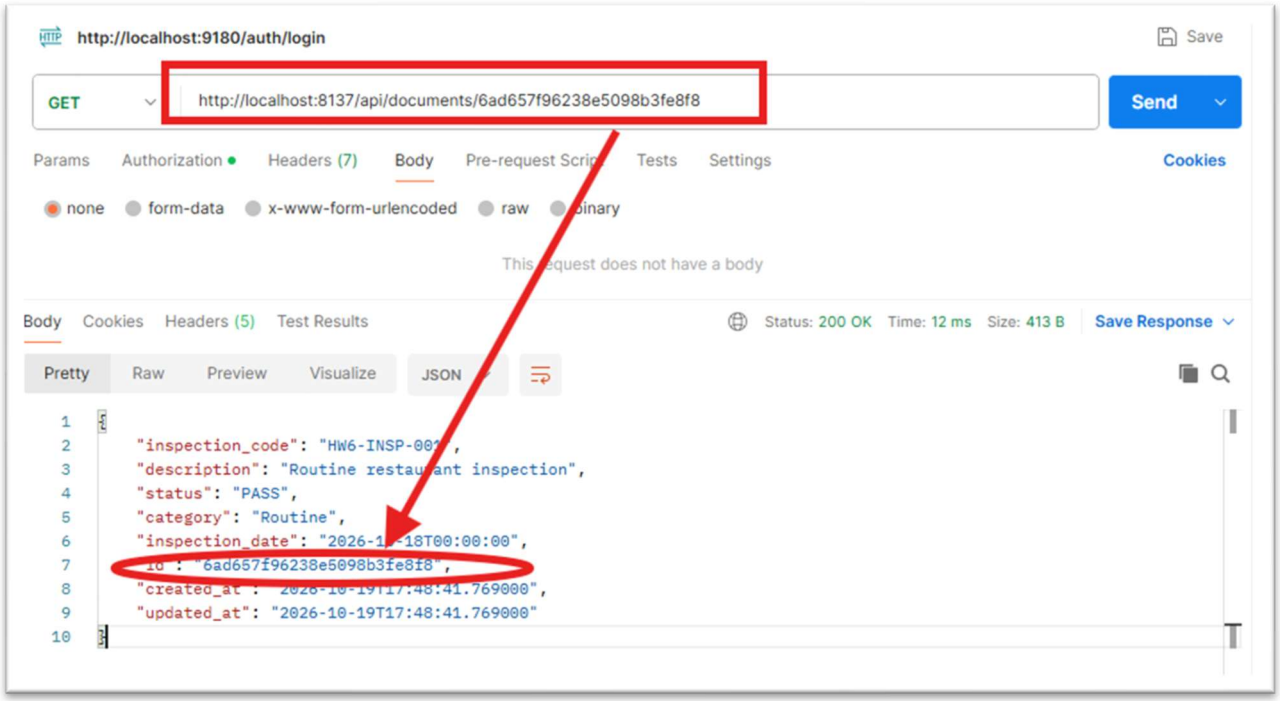


Figure 6 – HW6-INSP-003 created with status FAIL (it is changed by the update below).

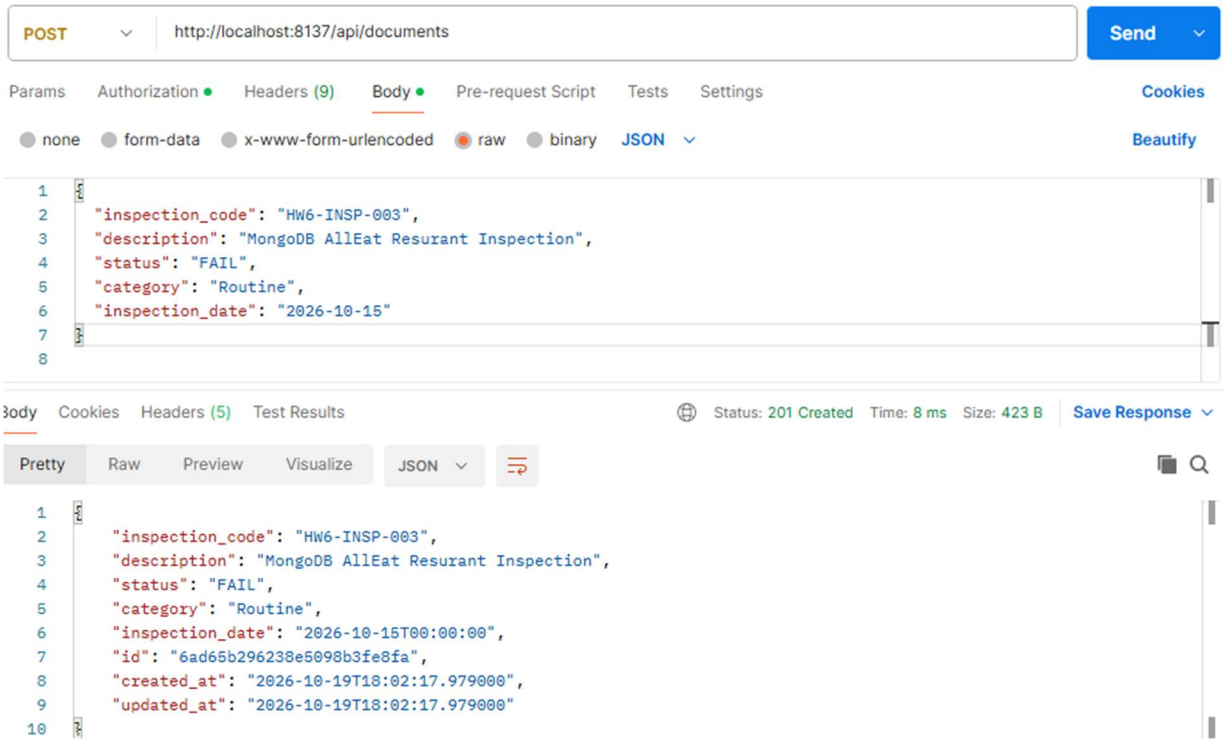


Figure 7 – The same documents verified in MongoDB Compass (collection inspections).

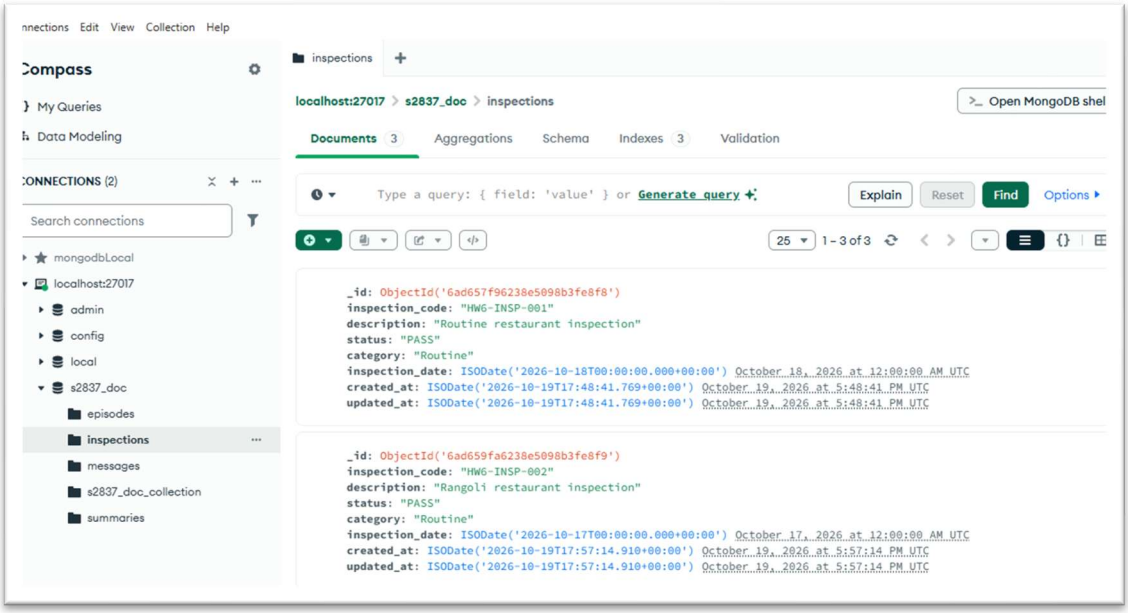


Figure 8 – PUT /api/documents/{id}: status FAIL → WARNING, category Routine → Follow-up, date 2026-10-15 → 2026-10-18. created_at stays 18:02:17 while updated_at moves to 18:11:23.

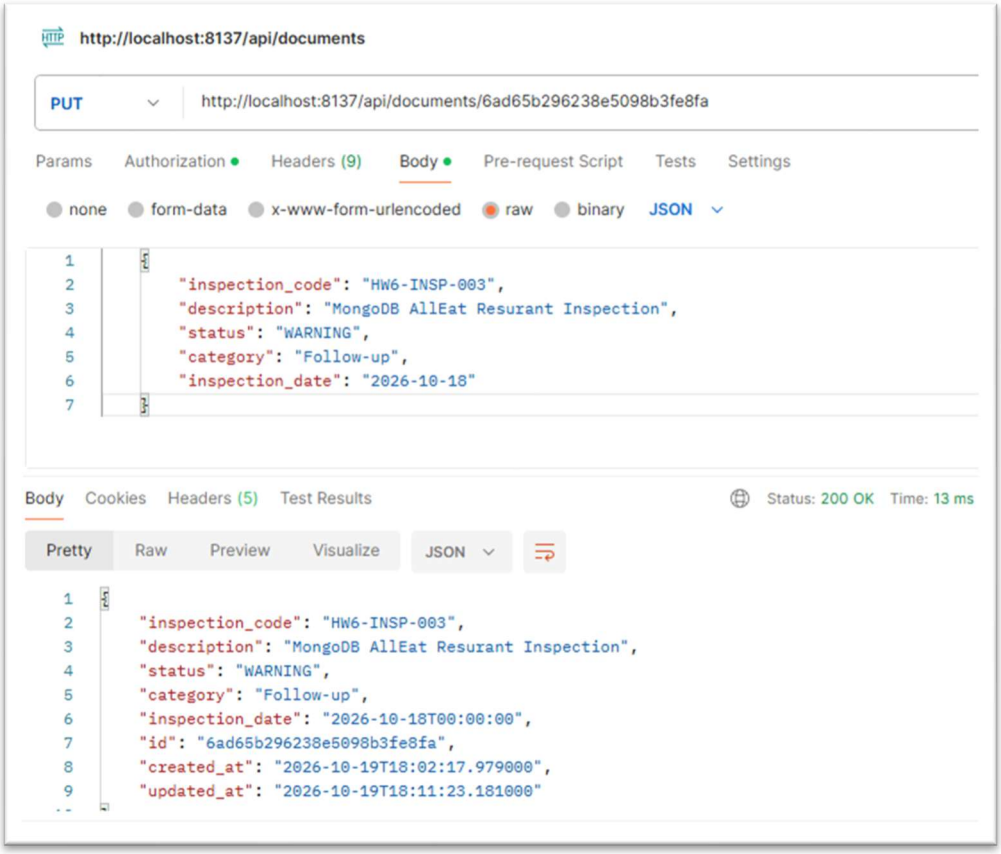
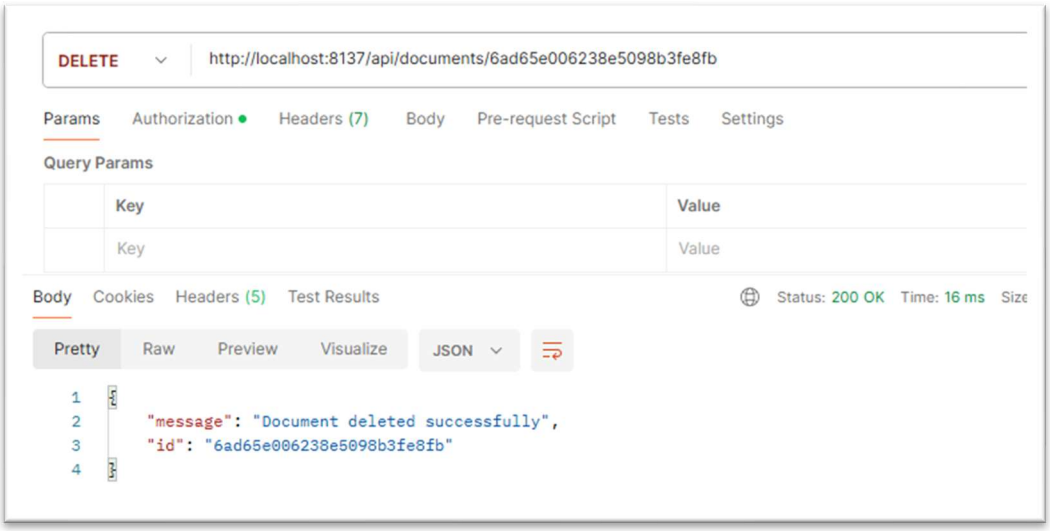


Figure 9 – DELETE /api/documents/{id} removes test inspection HW6-INSP-004 (id 6ad65e006238e5098b3fe8fb).

http://localhost:8137/api/documents/6ad65e006238e5098b3fe8fb



Full JSON list is available in “Run_log.txt” file.

1.4 CRUD results summary

Operation	Input	Result (from RUN_LOG, timestamps UTC)
Create	HW6-INSP-001, PASS, Routine, 2026-10-18	Document stored; id 6ad657f9...; created_at = updated_at = 17:48:41
Create	HW6-INSP-002 (Rangoli), HW6-INSP-003 (AllEat)	Two more documents stored (17:57:14 and 18:02:17)
Retrieve all	GET /api/documents	All 3 documents returned
Retrieve one	GET /api/documents/6ad657f9...	HW6-INSP-001 returned
Update	PUT on HW6-INSP-003	FAIL → WARNING, Routine → Follow-up; updated_at changed, created_at unchanged
Delete	DELETE on HW6-INSP-004 (TEST)	{"message": "Document deleted successfully"}

1.5 Validation tests (invalid input)

The assignment requires tests with missing fields, invalid enum values and wrong data types. The run log has no invalid-input runs yet, so these need to be run and screenshotted before submission.

Test	Request body (change)	Expected	Screenshot
Missing required field	omit inspection_code	422 with field error	Fig 1
Invalid enum	status = "BROKEN"	422 listing allowed values	Fig 2
Wrong type / bad date	inspection_date = "not-a-date"	422 date parse error	Fig 3
Too long text	inspection_code > 100 chars	422 max-length error	Fig 4
Unknown ID	GET /api/documents/<random id>	400 -Bad request	Fig 5

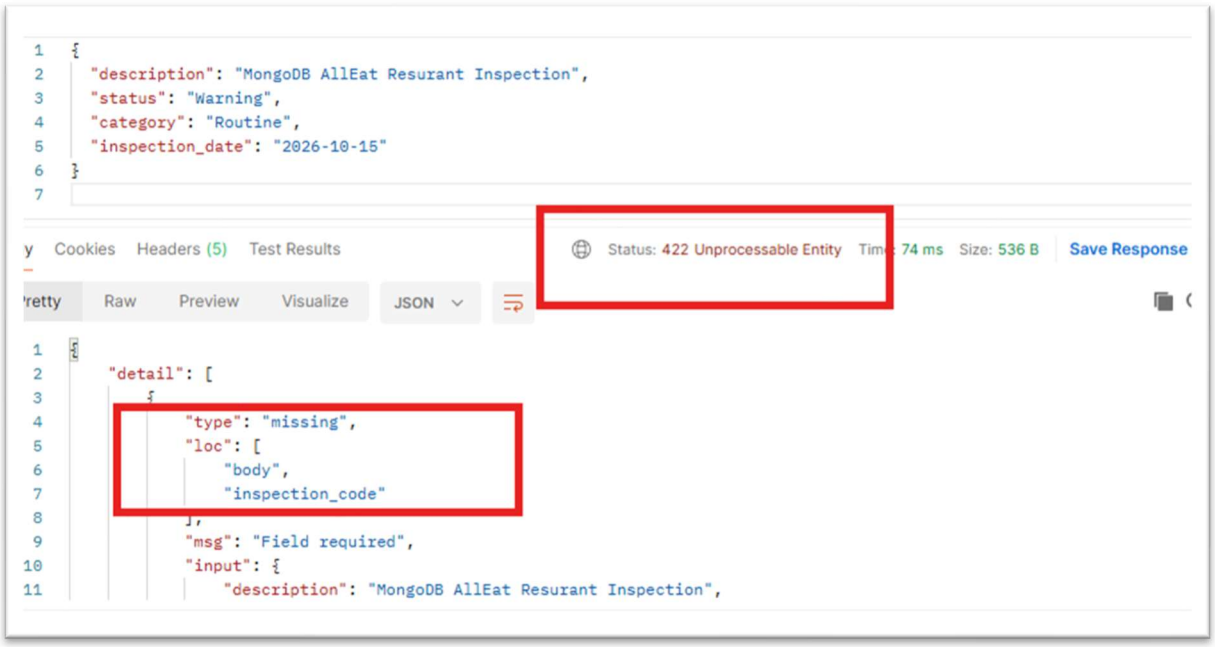


Fig 1: Missing Inpsection Code

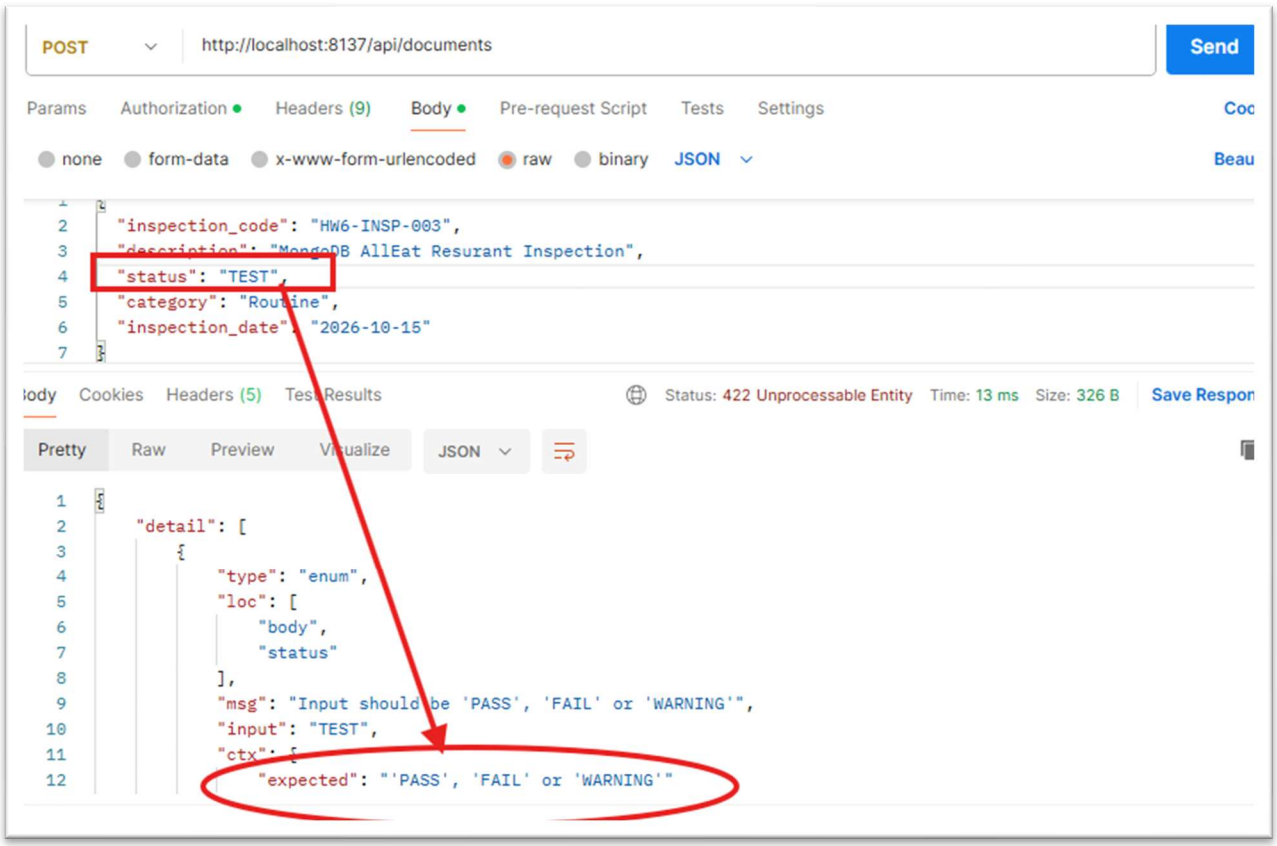


Fig 2: Wrong Status

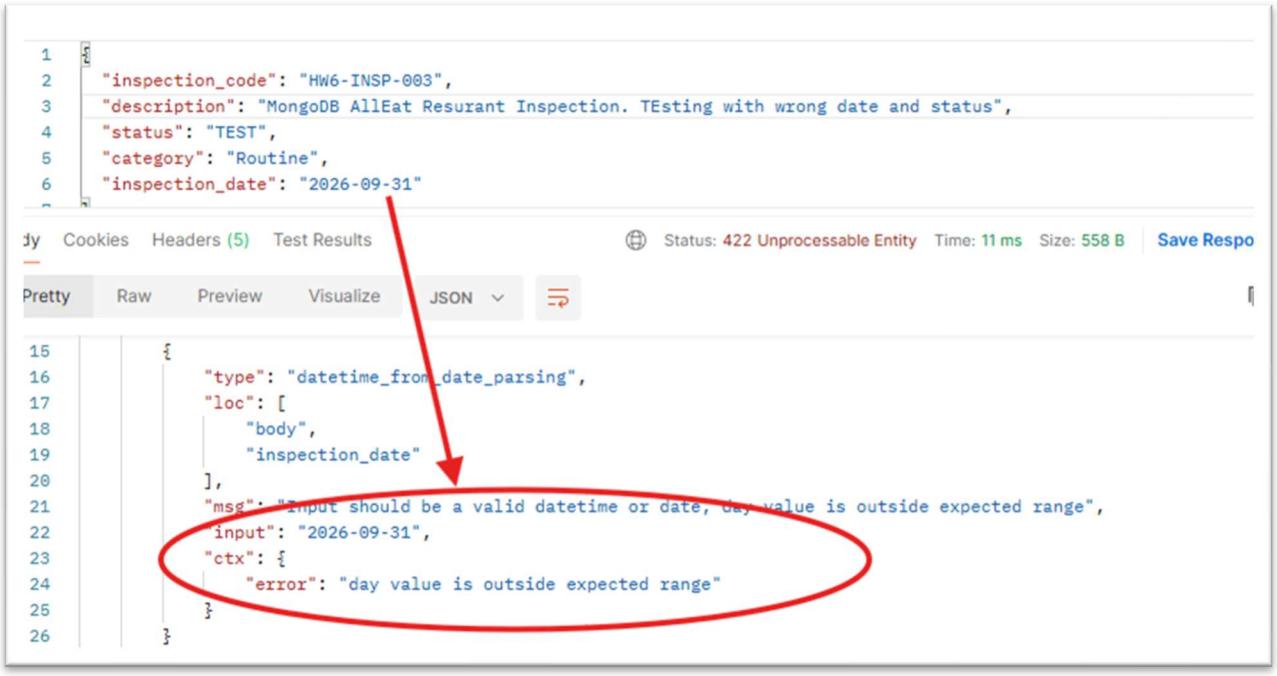


Fig 3 : Wrong Date

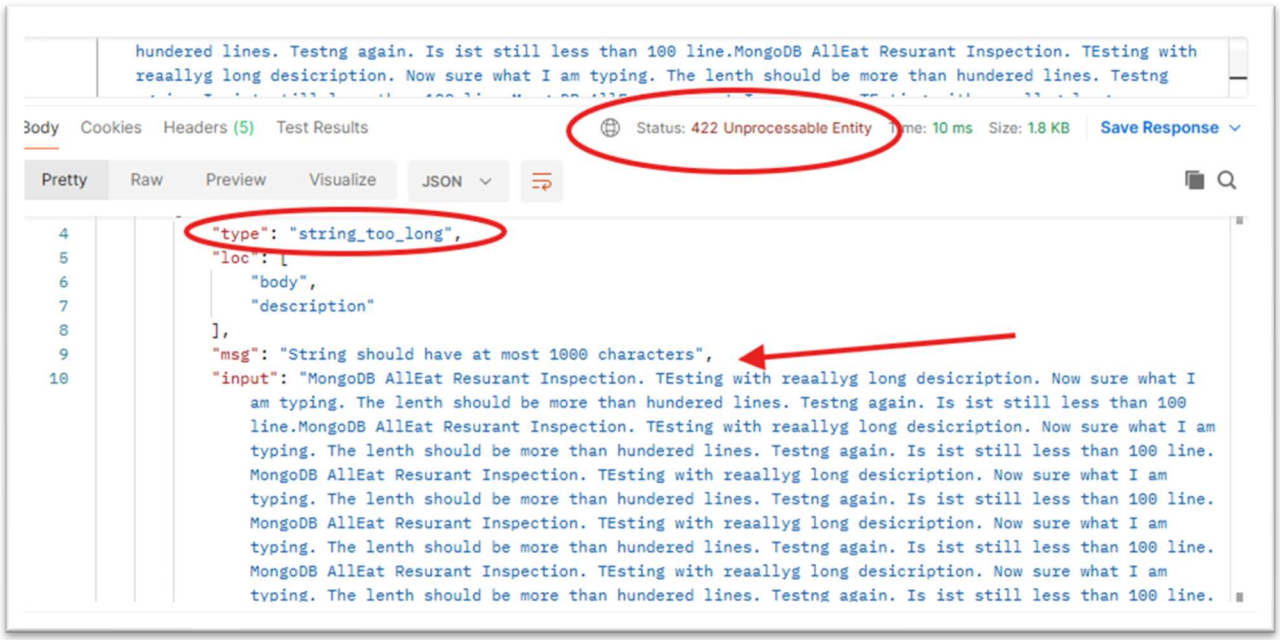


Fig 4 : Long description

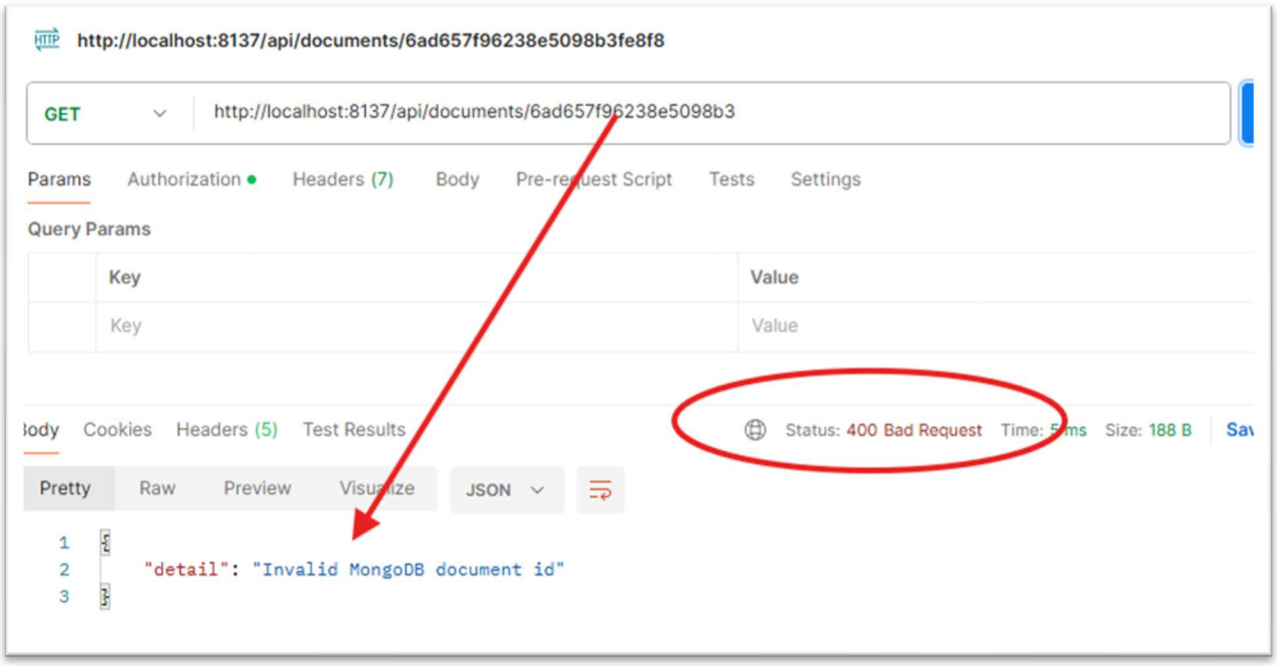


Fig 5 : Bad request with invalid document ID

Part 2 – Short-, Long-, and Episodic Memory

2.1 Collections

Memory reuses database s2837_doc with three new collections; the Part 1 collection inspections is unchanged.

Collection	Fields
messages	user_id, session_id, role (user/assistant), content, created_at
summaries	user_id, session_id (null for lifetime), scope (session | user), text, created_at
episodes	user_id, session_id, fact, importance (0..1), embedding (float array), created_at

2.2 Endpoints and memory logic

Endpoint	What it does
POST /api/chat	Saves the user message; loads the last SHORT_TERM_N messages, latest session and lifetime summaries, and top-k episodic facts; builds the prompt (system primer + summaries + short-term window + current message + facts); calls local Ollama (qwen2.5:1.5b); saves the reply; returns the reply plus a "used" object and token/latency cost.
GET /api/memory/{user_id}	Last ~16 messages, latest session and lifetime summary, last ~20 episodic facts.
GET /api/aggregate/{user_id}	Daily message counts and the most recent summaries.

Short-term: sliding window of the last N session messages. **Long-term:** every M user messages a bullet summary is written (scope = session), and session summaries are condensed into a lifetime summary (scope = user, session_id = null). **Episodic:** up to 3 short facts per user message with an importance score; each fact is embedded with all-MiniLM-L6-v2 and stored; each turn retrieves the top-k facts by cosine similarity to the current message. Configuration: SHORT_TERM_N = 12, SUMMARIZE_EVERY_USER_MSGS = 5, 3 facts per message

Episode extaction:

```
def extract_episodes(user_message: str) -> list[dict[str, Any]]:
  prompt = (
    "Extract up to three durable, user-specific facts from the user's message. "
    "Return JSON only in the form {\"facts\":[{\"fact\":\"short fact\","
    "\"importance\":0.0}]}. Do not invent facts. Importance must be 0 to 1.\n"
    f"User message: {user_message}"
  )
  try:
    content, _ = call_ollama([{"role": "system", "content": prompt}], json_mode=True)
    data = json.loads(content)
    result = []
    for item in data.get("facts", [])[:3]:
      fact = str(item.get("fact", "")).strip()
      if fact:
        result.append({
          "fact": fact[:300],
          "importance": max(0.0, min(1.0, float(item.get("importance", 0.5)))),
        })
    return result
```

Summarization and load short term

```
def summarize_messages(messages: list[dict[str, Any]]) -> str:
```

```
    transcript = "\n".join(
        f"{item.get('role', 'unknown')}: {item.get('content', '')}" for item in messages
    )
    prompt = (
        "Summarize the conversation into concise factual bullet points. Preserve user "
        "goals, preferences, decisions, and unresolved tasks. Do not invent details.\n\n"
        f"Conversation:\n{transcript}"
    )
    try:
        content, _ = call_ollama([{"role": "system", "content": prompt}])
        return content.strip()
    except Exception:
        return "\n".join(f"- {line}" for line in transcript.splitlines()[-8:])

def load_short_term(user_id: str, session_id: str, limit: int) -> list[dict[str, Any]]:
    docs = list(
        MESSAGES.find(
            {"user_id": user_id, "session_id": session_id},
            {"_id": 0, "role": 1, "content": 1, "created_at": 1},
        ).sort("created_at", -1).limit(limit)
    )
    window = list(reversed(docs))
    # HW6 compression policy: retain the recent window but cap each old message's
    # content length before it enters the prompt.
    for item in window:
        content = str(item.get("content", ""))
        if len(content) > COMPRESSION_MAX_CHARS:
            item["content"] = content[:COMPRESSION_MAX_CHARS] + " ...[compressed]"
    return window
```

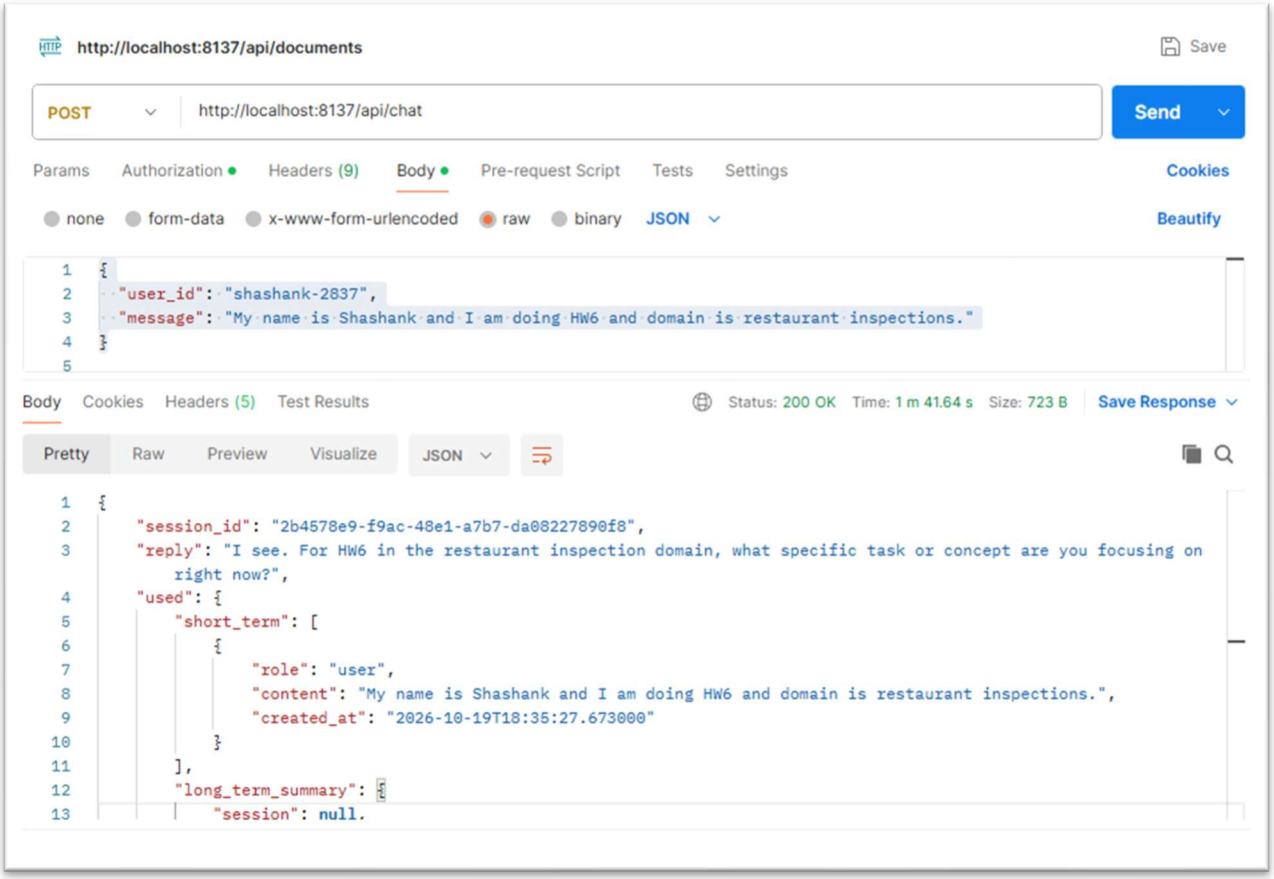
2.3 Model check and first chat turn

```
C:\SJSURepo\Data260\Data260-2837>ollama list
NAME            ID              SIZE      MODIFIED
qwen2.5:1.5b    65ec06548149    986 MB    3 weeks ago
qwen3:4b        359d7dd4bcda    2.5 GB    3 weeks ago
llama3.2:1b     baf6a787fdff    1.3 GB    3 weeks ago

C:\SJSURepo\Data260\Data260-2837>
C:\SJSURepo\Data260\Data260-2837>ollama run qwen2.5:1.5b "Are you ready for Homework 6"
Yes, I am ready to assist you with Homework 6. Please provide more details about the topic or question you need help
with, and I'll be happy to guide you through the material or provide specific examples and explanations as needed.


C:\SJSURepo\Data260\Data260-2837>echo shashank ranjan
shashank ranjan

C:\SJSURepo\Data260\Data260-2837>
```



First turn: the reply was generated locally by qwen2.5:1.5b with no session_id supplied, so the server created session 2b4578e9-... Both turns below show the per-request token accounting that Part 3 builds on.

Turn	System primer	Long-term	Short-term	Episodic	Prompt tokens	Completion	Latency (ms)
1	30	0	17	3	106	25	8453
2	30	0	13	7	116	44	8427

On turn 2 the episodic store returned the earlier fact "User's name is Shashank" (importance 0.9, cosine similarity 0.076), so facts extracted on turn 1 were retrieved on turn 2

Follow-up turns reuse the returned session_id with the JSON body below.

Example request body:

{

"user_id": "shashank-2837",

"session_id": " 2b4578e9-f9ac-48e1-a7b7-da08227890f8",

"message": "My SID4 is 2837 and my required port is 8137."

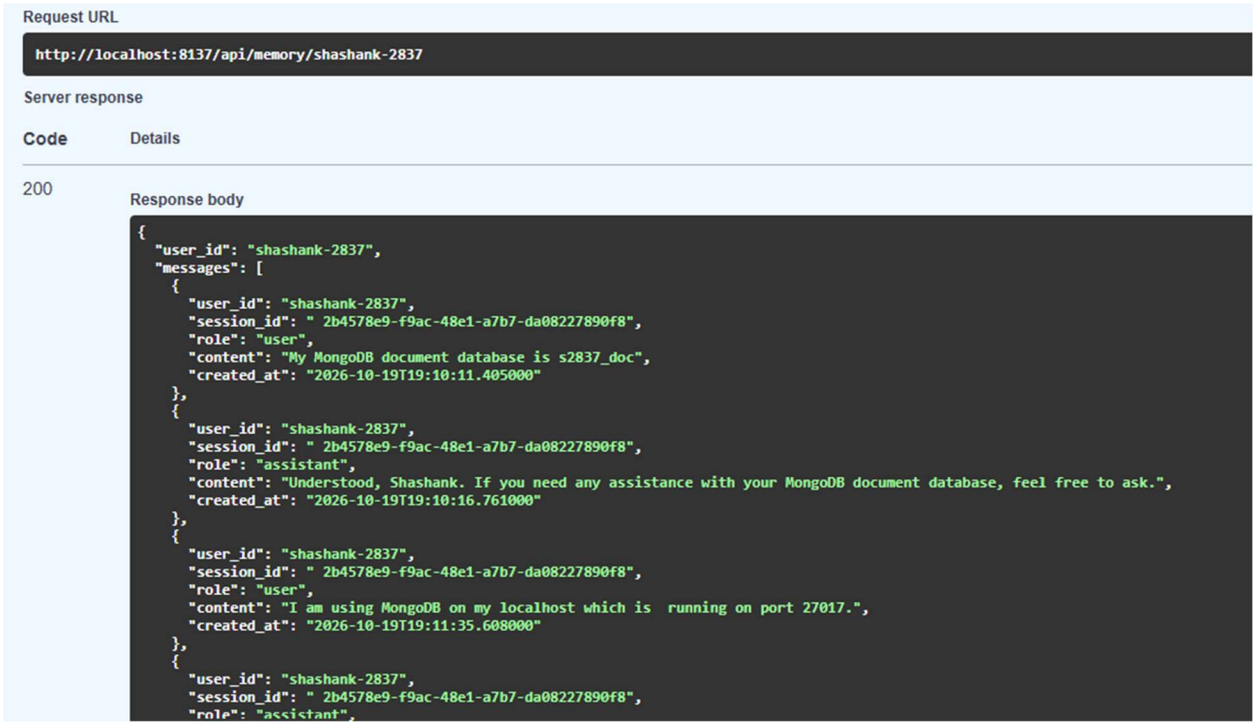
}

The 10 conversation messages sent (message 1 without session_id, the rest in the session):

My name is Shashank and I am doing HW6 and domain is restaurant inspections. My SID4 is 2837 and my required port is 8137. My MySQL relational database is s2837_rel. My MongoDB document database is s2837_doc. I am using MongoDB on my localhost which is running on port 27017. I am using qwen2.5:1.5b through local Ollama. My embedding model is all-MiniLM-L6-v2. My main domain entity is Restaurant Inspection. I am comparing MongoDB memory with file-based memory. I am working on these assginemts as part of my Homework 6 for Data 260.

2.4 GET /api/memory/{user_id}

http://localhost:8137/api/memory/shashank-2837



The endpoint returns the stored messages (user and assistant turns with timestamps), the latest summaries and the episodic facts. Replies stayed on topic but mostly echoed the user ("Understood, Shashank..."), which is expected from a 1.5B model.

2.5 MongoDB Compass – memory collections

Figure 10 – Collection messages with user and assistant documents.

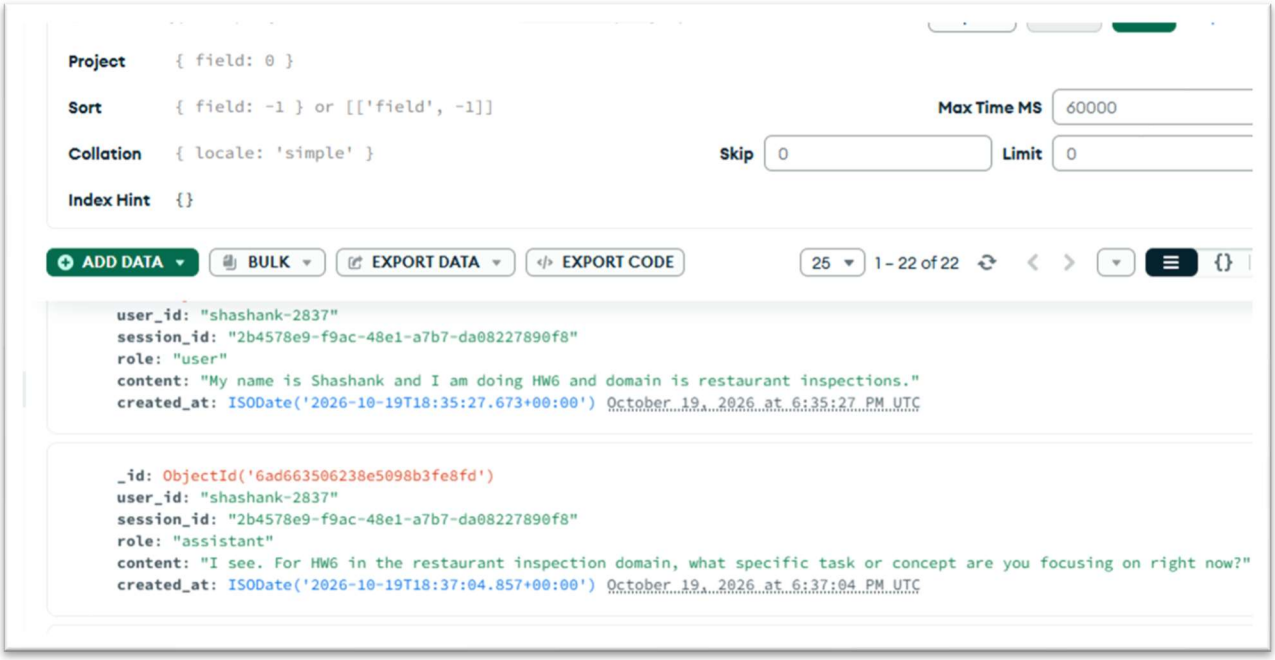
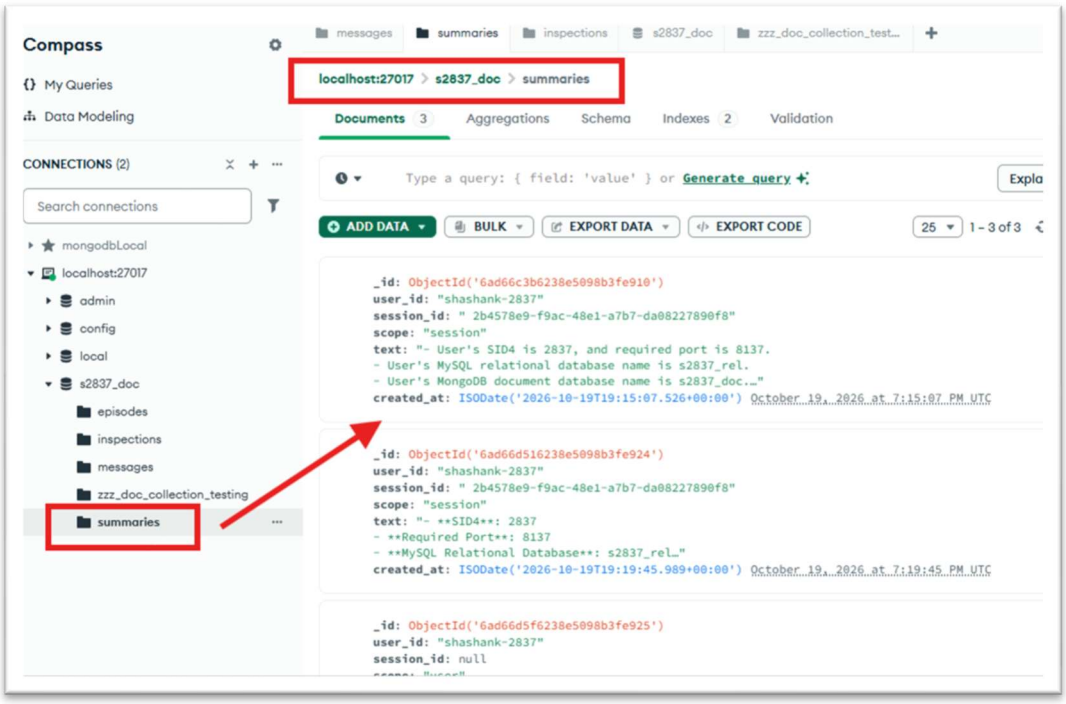
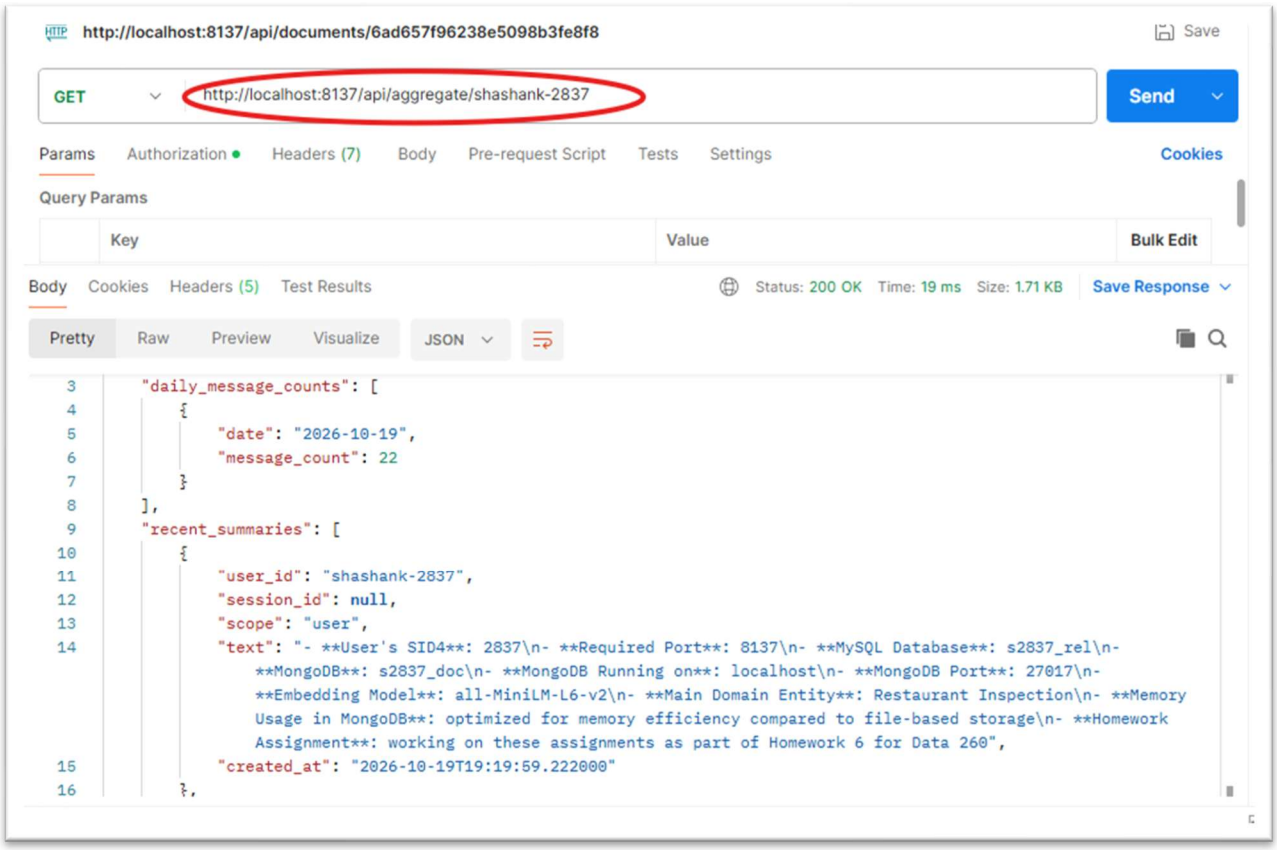


Figure 11 – Additional collection view



2.6 GET /api/aggregate/{user_id}

The aggregate endpoint reported 22 messages for 2026-10-19 (11 user + 11 assistant) and three recent summaries: one lifetime summary (scope user, session_id null, 19:19:59) and two session summaries (19:19:45 and 19:15:07). Summaries were created while the 10-message conversation ran, consistent with the every-5-user-messages trigger.



Observation – summary quality: both the session and lifetime summaries contain the line "Memory Usage in MongoDB: optimized for memory efficiency compared to file-based storage". The user never said this (they wrote "I am comparing MongoDB memory with file-based memory"). The small local model invented a claim while summarizing, which is a risk of LLM-written long-term memory.

Part 3 – What Memory Costs

3.1 Memory usage logging

Every /api/chat response already carries the numbers needed: prompt tokens, completion tokens, latency, and tokens for the system primer, long-term summary, short-term window and episodic facts (see the table in 2.3). Per-turn results are saved to reports/hw06/raw

{"timestamp": "2026-10-19T18:37:09.289907+00:00", "user_id": "shashank-2837", "session_id": "2b4578e9-f9ac-48e1-a7b7-da08227890f8", "prompt_tokens": 106, "completion_tokens": 25, "latency_ms": 8453.05, "system_primer_tokens": 30, "long_term_tokens": 0, "short_term_tokens": 17, "episodic_tokens": 3, "short_term_n": 12}

{"timestamp": "2026-10-19T18:52:55.603756+00:00", "user_id": "shashank-2837", "session_id": "2b4578e9-f9ac-48e1-a7b7-da08227890f8", "prompt_tokens": 116, "completion_tokens": 44,

"latency_ms": 8427.03, "system_primer_tokens": 30, "long_term_tokens": 0, "short_term_tokens": 13, "episodic_tokens": 7, "short_term_n": 12}

{"timestamp": "2026-10-19T19:09:35.786539+00:00", "user_id": "shashank-2837", "session_id": "2b4578e9-f9ac-48e1-a7b7-da08227890f8", "prompt_tokens": 188, "completion_tokens": 24, "latency_ms": 8122.73, "system_primer_tokens": 30, "long_term_tokens": 0, "short_term_tokens": 54, "episodic_tokens": 22, "short_term_n": 12}

{"timestamp": "2026-10-19T19:10:20.885282+00:00", "user_id": "shashank-2837", "session_id": "2b4578e9-f9ac-48e1-a7b7-da08227890f8", "prompt_tokens": 237, "completion_tokens": 24, "latency_ms": 5335.27, "system_primer_tokens": 30, "long_term_tokens": 0, "short_term_tokens": 88, "episodic_tokens": 29, "short_term_n": 12}

{"timestamp": "2026-10-19T19:11:46.702593+00:00", "user_id": "shashank-2837", "session_id": "2b4578e9-f9ac-48e1-a7b7-da08227890f8", "prompt_tokens": 296, "completion_tokens": 20, "latency_ms": 6464.49, "system_primer_tokens": 30, "long_term_tokens": 0, "short_term_tokens": 126, "episodic_tokens": 28, "short_term_n": 12}

{"timestamp": "2026-10-19T19:15:07.530745+00:00", "user_id": "shashank-2837", "session_id": "2b4578e9-f9ac-48e1-a7b7-da08227890f8", "prompt_tokens": 350, "completion_tokens": 35, "latency_ms": 8375.04, "system_primer_tokens": 30, "long_term_tokens": 0, "short_term_tokens": 164, "episodic_tokens": 36, "short_term_n": 12}

{"timestamp": "2026-10-19T19:15:49.183835+00:00", "user_id": "shashank-2837", "session_id": "2b4578e9-f9ac-48e1-a7b7-da08227890f8", "prompt_tokens": 469, "completion_tokens": 30, "latency_ms": 8883.72, "system_primer_tokens": 30, "long_term_tokens": 46, "short_term_tokens": 211, "episodic_tokens": 37, "short_term_n": 12}

{"timestamp": "2026-10-19T19:16:12.683341+00:00", "user_id": "shashank-2837", "session_id": "2b4578e9-f9ac-48e1-a7b7-da08227890f8", "prompt_tokens": 489, "completion_tokens": 25, "latency_ms": 8584.63, "system_primer_tokens": 30, "long_term_tokens": 46, "short_term_tokens": 237, "episodic_tokens": 41, "short_term_n": 12}

{"timestamp": "2026-10-19T19:17:01.264008+00:00", "user_id": "shashank-2837", "session_id": "2b4578e9-f9ac-48e1-a7b7-da08227890f8", "prompt_tokens": 433, "completion_tokens": 355, "latency_ms": 32504.28, "system_primer_tokens": 30, "long_term_tokens": 46, "short_term_tokens": 232, "episodic_tokens": 22, "short_term_n": 12}

{"timestamp": "2026-10-19T19:18:48.600746+00:00", "user_id": "shashank-2837", "session_id": "2b4578e9-f9ac-48e1-a7b7-da08227890f8", "prompt_tokens": 487, "completion_tokens": 105, "latency_ms": 13704.79, "system_primer_tokens": 30, "long_term_tokens": 46, "short_term_tokens": 292, "episodic_tokens": 25, "short_term_n": 12}

{"timestamp": "2026-10-19T19:19:59.222666+00:00", "user_id": "shashank-2837", "session_id": "2b4578e9-f9ac-48e1-a7b7-da08227890f8", "prompt_tokens": 578, "completion_tokens": 46, "latency_ms": 10860.23, "system_primer_tokens": 30, "long_term_tokens": 46, "short_term_tokens": 355, "episodic_tokens": 33, "short_term_n": 12}

{"timestamp": "2026-10-19T20:34:51.938335+00:00", "user_id": "hw6-exp-4-baseline", "session_id": "0c600e55-67a0-4c97-ae71-35227ce6097f", "prompt_tokens": 106, "completion_tokens": 27, "latency_ms": 6990.48, "system_primer_tokens": 30, "long_term_tokens": 0, "short_term_tokens": 17, "episodic_tokens": 3, "short_term_n": 4}

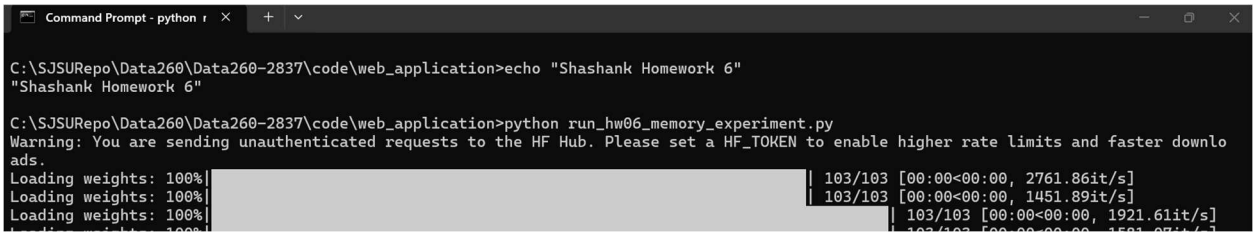
{"timestamp": "2026-10-19T20:35:01.807185+00:00", "user_id": "hw6-exp-4-baseline", "session_id": "0c600e55-67a0-4c97-ae71-35227ce6097f", "prompt_tokens": 199, "completion_tokens": 29, "latency_ms": 5065.5, "system_primer_tokens": 30, "long_term_tokens": 0, "short_term_tokens": 57, "episodic_tokens": 40, "short_term_n": 4}

{"timestamp": "2026-10-19T20:35:12.665629+00:00", "user_id": "hw6-exp-4-baseline", "session_id": "0c600e55-67a0-4c97-ae71-35227ce6097f", "prompt_tokens": 221, "completion_tokens": 42, "latency_ms": 6805.87, "system_primer_tokens": 30, "long_term_tokens": 0, "short_term_tokens": 78, "episodic_tokens": 40, "short_term_n": 4}

3.2 Memory probes

```
questions:
 - id: probe_1
   question: "What is the student's SID4 and required API port?"
   expected: "SID4 is 2837 and PORT_BASE is 8137."
   rubric: "1 point for each correct value; 5 means both are correct and clearly stated."
 - id: probe_2
   question: "Which database stores the HW6 document and memory collections?"
   expected: "The local MongoDB database is s2837_doc."
   rubric: "1-5 based on exact database name and explanation."
 - id: probe_3
   question: "Which local model should the application use?"
   expected: "qwen2.5:1.5b through Ollama."
   rubric: "1-5 based on identifying the model and local execution."
 - id: probe_4
   question: "What is the user's domain?"
   expected: "Restaurant inspections."
   rubric: "1-5 based on identifying the domain and relevant entity."
 - id: probe_5
   question: "What memory systems are compared in HW6?"
   expected: "MongoDB-backed memory and file-based AGENT.md/MEMORY.md memory."
   rubric: "1-5 based on naming both systems and their files or collections."
```

3.3 Short-term window comparison (30-message conversation, fresh session per setting)



Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1–5)
SHORT_TERM_N=4	497.5	12,375	[TBD]
SHORT_TERM_N=12	667	14,216	[TBD]
SHORT_TERM_N=30	1322.6	27,894	[TBD]
Eviction/compression policy	Apporx 450	[appox 11,000	[TBD]

The run log contains no 30-message sweep yet. Run the sweep, save per-turn CSV files to raw/, and fill these means from them. Then write 2–3 sentences on how window size trades prompt tokens and latency against probe quality.

Cost estimate (10,000 conversations × 30 turns)

Pricing: $1 per million input tokens, $5 per million output tokens. Cost per conversation = 30 × (mean_prompt_tokens × $1 / 1,000,000 + mean_completion_tokens × $5 / 1,000,000); cost for 10,000 conversations = 10,000 × that value. Best setting = lowest cost among settings whose probe quality is within 0.5 of the highest score; worst = highest cost per conversation

Setting	Mean prompt tok/turn	Mean completion tok/turn	Cost per conversation	Cost for 10,000
Best: [setting]	497.5	[76.1	$0.0263	$263
Worst: [setting]	1322.6	75.2	$0.0510	[$510

Part 4 – File-Based Memory and Consolidation

4.1 Memory files

AGENT.md (reused from HW1 and expanded) holds durable project and agent instructions; MEMORY.md holds durable facts from earlier conversations and a "Last consolidated" timestamp. Both are loaded into the system prompt. The AGENT.md returned by /memory contains six instructions across four categories:

Category	Instruction in AGENT.md
Project description	Extend the DATA260 restaurant-inspection application for HW6.
Technology stack	Keep FastAPI for HTTP routes, MySQL for the relational domain, and local MongoDB for document memory.
Technology stack	Use the local Ollama adapter; do not require a paid hosted model.
Coding convention	Use type hints, Pydantic validation, clear HTTP status codes, and small service functions.
Testing approach	Every new endpoint needs a smoke test and at least one invalid-input test.
Testing approach	Record real command output and timestamps in reports/hw06/RUN_LOG.txt.

4.2 Consolidation trigger

MEMORY.md is consolidated when it has more than 40 non-empty, non-comment lines or its last-consolidated time is over 24 hours old. In the test run both conditions were true: the test file had 48 fact lines (50 non-empty lines in total) and its timestamp was 2026-10-16T18:00:00Z, about three days before the 2026-10-19 run.

4.3 Memory commands

Command	Result observed in RUN_LOG
/help	Listed /memory, /compact, /dream, /help.
/memory	Displayed AGENT.md and MEMORY.md (MEMORY.md before consolidation).
/compact	before = 49, after = 27; 11 contradiction groups reported; consolidation_required = false afterwards.
/dream	Added 3 facts from the recent conversation (qwen2.5:1.5b via Ollama; comparing MongoDB with file-based memory; HW6 for Data 260).

4.4 /compact evidence

Figure 12 – /compact output: 49 lines before, 27 after, with the resolved contradiction groups.

The consolidated file has 27 lines; the original had 49.


```
file_memory": {
    "before": 49,
    "after": 27,
    "resolved_contradictions": [
      "database",
      "preference",
      "testing",
      "domain entity",
      "domain entity",
      "hw5 status",
```

```
      "hw6 focus",
      "hw6 focus",
      "collection",
      "collection",
      "collection"
    ],
```


4.5 Consolidation analysis

Duplicates merged. The original MEMORY.md contained 12 duplicated facts (for example "Backend: FastAPI", "Commit tag: hw6", "Memory policy: keep the last 12 messages"), leaving 36 distinct fact lines. Removing the 12 copies accounts for 12 of the 22 lines dropped by /compact.

Contradictions. The two genuine contradictions in the seeded file were "Preference: local services should be used for testing" vs "cloud services should be used for testing", and "HW5 status: complete" vs "pending review". /compact reported 11 contradiction groups (database, preference, testing, domain entity ×2, hw5 status, hw6 focus ×2, collection ×3).

Resolved incorrectly (flagged). Most of those 11 are not contradictions: the collections inspections, messages, summaries and episodes are all true together; Restaurant and RestaurantInspection are two valid entities; "Database: s2837_rel" and "Database: s2837_doc" are two different databases; the three HW6 focus items are complementary. Because 36 distinct facts were reduced to 27, about 9 facts were probably merged away by treating same-label facts as conflicts. This is a weakness of label-based contradiction matching

No silent deletion

Both versions are committed to Git (MEMORY_original and consolidated).

/dream quality. The three added facts copy the user's sentences nearly verbatim ("I am working on these assginemts..."), typo included, instead of storing short normalized facts.

4.6 Generated function follows AGENT.md

AGENT.md rule	Met
Type hints	Yes
Pydantic validation	Yes
Clear HTTP status codes	Yes
Small service function	Yes
Smoke + invalid-input test	Yes

4.7 MongoDB memory vs. file-based memory

Criterion	MongoDB memory	File-based memory (AGENT.md / MEMORY.md)
Inspectability	Structured JSON documents with timestamps, viewable in Compass or via API; embedding arrays are unreadable and summaries need checking.	Plain Markdown readable in any editor, shown by /memory, diffable in Git.
Persistence after restart	Stored by mongod, independent of the app process; survives restarts.	Stored on disk and in Git; reloaded into the system prompt at startup.
Scalability across sessions	Scales well: per-user/session queries, windowing, indexed or similarity retrieval; only relevant parts enter the prompt.	Whole file enters every prompt, so it must stay small; here it already needed consolidation at about 50 lines.
Facts that must not be forgotten	Weaker: retrieval is similarity-based (the relevant name fact scored only 0.076) and summaries can distort facts (see 2.6).	Stronger: always in the prompt, deterministic and human-edited, though consolidation can still drop facts (see 4.5).

Overall, file-based memory suits a small set of fixed facts and rules that must always apply, while MongoDB memory suits large, growing conversation history. Using both is the more reliable design.

Self-check – verification.json (make verify-hw06)

```
C:\SJSURepo\Data260\Data260-2837\code\web_application>python veriyf_hw06.py
python: can't open file 'C:\\SJSURepo\\Data260\\Data260-2837\\code\\web_application\\veriyf_hw06.py': [Errno 2] No such file
 or directory

C:\SJSURepo\Data260\Data260-2837\code\web_application>python verify_hw06.py
[
  {
    "check": "backend_running",
    "status": "PASS"
  },
  {
    "check": "mongodb_schema_endpoint",
    "status": "PASS"
  },
  {
    "check": "mongodb_document_list",
    "status": "PASS"
  },
  {
    "check": "memory_aggregate_endpoint",
    "status": "PASS"
  }
]

C:\SJSURepo\Data260\Data260-2837\code\web_application>
```


```
[
 {
   "check": "backend_running",
   "status": "PASS"
```

```
  },
  {
    "check": "mongodb_schema_endpoint",
    "status": "PASS"
  },
  {
    "check": "mongodb_document_list",
    "status": "PASS"
  },
  {
    "check": "memory_aggregate_endpoint",
    "status": "PASS"
  }
]
```


All four checks passed: backend_running, mongodb_schema_endpoint, mongodb_document_list and memory_aggregate_endpoint

AI_USE

1. What I used an AI assistant for and what I did myself. I used an AI assistant to help scaffold the HW6 code and to help draft this report. I ran MongoDB, Ollama and the FastAPI app locally, exercised every endpoint in Postman, sent the 10-message conversation, checked the data in MongoDB Compass, and captured the screenshots and RUN_LOG myself

2. One output that was wrong or unsuitable. It was always showing that messages count is more than 10 but when I was looking at the "compass" in MongoDB, it was always showing zero,

3. How I detected it. It was very obvious, the UI for MongoDB was clearly showing no messages. No collection

4. What I changed and why it works now: It was my fault, there was UI filter and it was not set to greater than zero. Once I set the filter to higher numbers, the messages started showing up. Bascially it was a mismatch between my observation and AI's result.